CSE 31
Computer Organization

Lecture 12 – Integer Representation

Announcements

- Lab
  - Lab 4 due this week (with 7 days grace period* after due date)
    - » Demo is REQUIRED to receive full credit**
  - Lab 5 out *next* week
    - » Due at 11:59pm on the same day of your lab after next (with 14 days grace period^ after due date)
    - » You must demo your submission to your TA within **21 days from posting of lab**
    - » Demo is REQUIRED to receive full credit**
- Reading and Homework assignments
  - Reading 03 (zyBooks 3.1 – 3.7, 3.9) due, 09-MAR
  - Homework 02 (zyBooks 2.1 – 2.9) due, 09-MAR
    - » Complete Participation Activities in each section to receive grade towards Reading assignments (part of course participation grades)
    - » Complete Challenge Activities in each section to receive grade towards Homework assignments
    - » IMPORTANT: Make sure to submit score to CatCourses by using the link provided on CatCourses

* There is a 10% penalty for late submissions during the grace period

^ There is NO PENALTY for submissions during the grace period for this assignment

** Submissions without demo will only receive partial credit for the TPS Activity portion of the assignment

Announcements

- Project 01
  - Due Fri, 20-MAR
  - Can work in teams of 2 students
    - » Each team member must identify teammate in "Comments…" text-box at the submission page
    - » If working in teams, each student must submit code (can be the same as teammate) and demo individually
    - » Grade can vary among teammates depending on demo
  - Demo required for project grade
    - » No partial credit for submission without demo
  - No grace period
    - » Must complete submission and demo by due date
- Midterm 01
  - On 12-MAR during class
  - See Announcements 07 and 08 on CatCourses for details

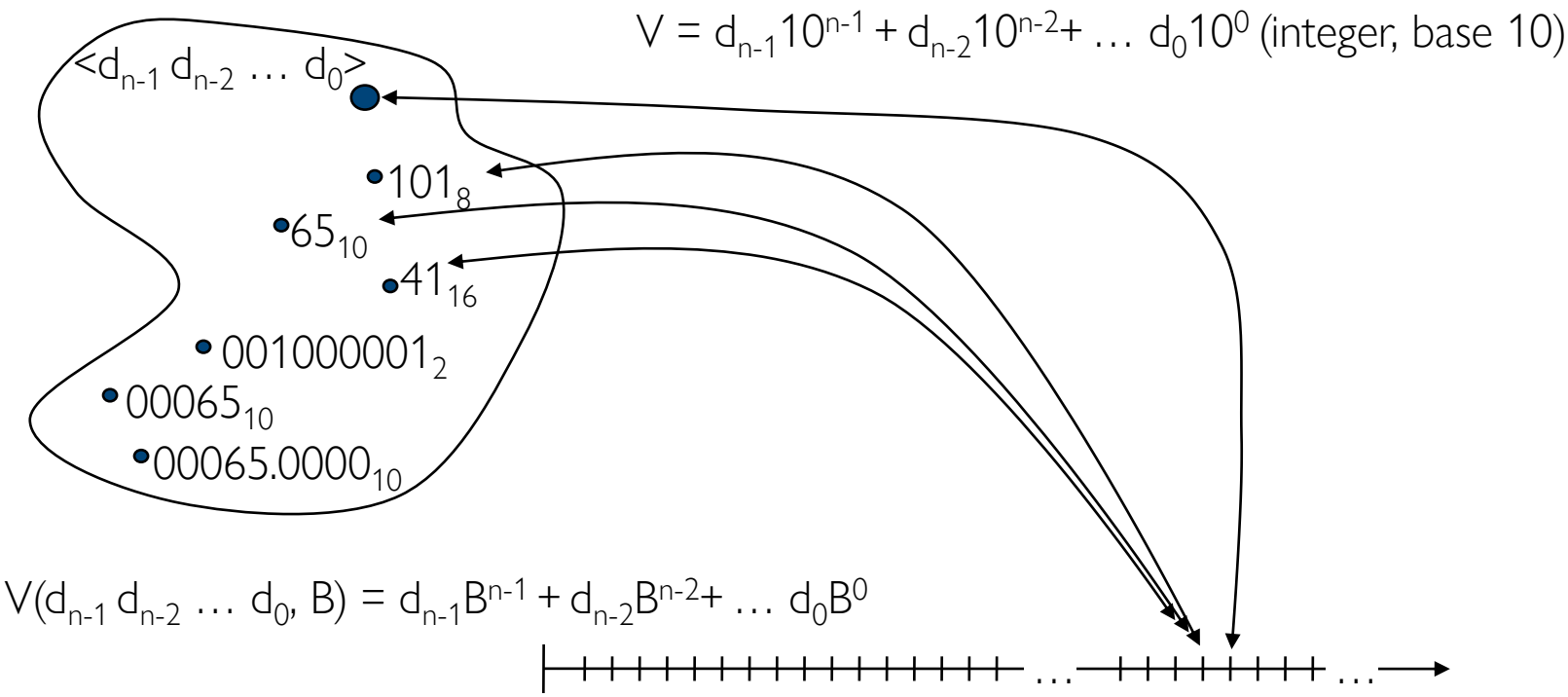
Number Representations

- What does this number mean?
  - 101

- Depends on what representation!

Representation and Meaning

- Objects are represented as collections of symbols (bits, digits)
- Their meaning is derived from what you do with them.



Representation (how many bits?)

- Characters?
  - 26 letters → 5 bits ($2^5 = 32$)
  - upper/lower case + punctuation
    → 7 bits (in 8 bits) ("ASCII")
  - standard code to cover all the world's languages → 8,16,32 bits ("Unicode") www.unicode.com

- Logical values?
  - 0 → False, 1 → True

- Color? Ex: *Red (00)* *Green (01)* *Blue (11)*

- Remember: N bits → at most ?? things

Representation (how many bits?)

- Characters?
  - 26 letters → 5 bits ($2^5 = 32$)
  - upper/lower case + punctuation
    → 7 bits (in 8 bits) ("ASCII")
  - standard code to cover all the world's languages → 8,16,32 bits ("Unicode") www.unicode.com

- Logical values?
  - 0 → False, 1 → True

- Color? Ex: *Red (00)* *Green (01)* *Blue (11)*

- Remember: N bits → at most 2^N things

How many bits to represent π ?

a) 1

b) 9 (π = 3.14, so that's 011 . 001 100)

c) 64 (Since modern computers are 64-bit machines)

d) Every bit the machine has!

e) ∞

We are going to learn how to represent floating point numbers later!

What to do with representations of numbers?

- Just what we do with numbers!
  - Add them
  - Subtract them
  - Multiply them
  - Divide them
  - Compare them

```
      1  1
      1  0  1  0
+     0  1  1  1
-----------------
   1  0  0  0  1
```

- Example: 10 + 7 = 17
  - …so simple to add in binary that we can build circuits to do it!
  - subtraction just as you would in decimal
  - Comparison: How do you tell if X > Y ?

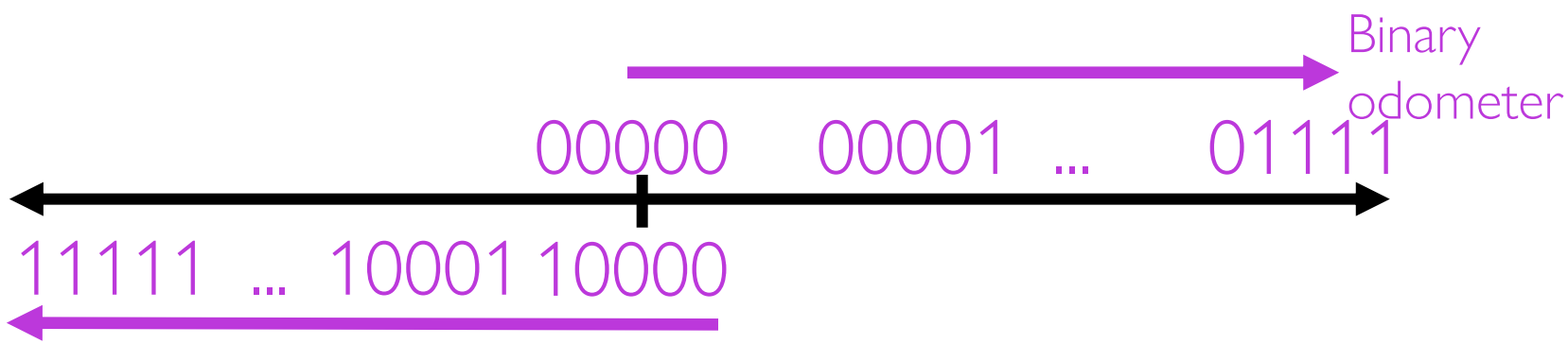
What if too big?

- Binary bit patterns are simply representatives of numbers. Strictly speaking they are called "numerals"
- Numbers really have an ∞ number of digits
  - with almost all being same (00…0 or 11…1) except for a few of the rightmost digits
  - Just don't normally show leading digits
- If result of add (or -, *, /) cannot be represented by these rightmost HW bits, ***overflow*** is said to have occurred.

Negative Numbers

- So far, *unsigned numbers*

- Obvious solution: define leftmost bit to be sign!
  - 0 → + , 1 → –
  - Rest of bits can be numerical value of number
- Representation called *sign and magnitude*



Shortcomings of Sign Magnitude?

- Arithmetic circuit complicated
  - Special steps depending whether signs are the same or not

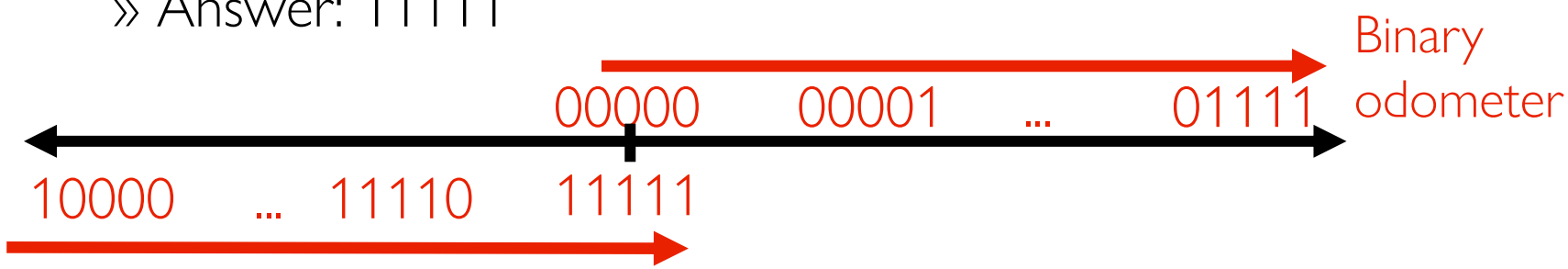
- Also, two zeros
  - $0\times00000000 = +0_{ten}$
  - $0\times80000000 = -0_{ten}$
  - What would two 0s mean for programming?

- Also, incrementing "binary odometer", sometimes increases values, and sometimes decreases!

- Therefore, sign and magnitude abandoned

Another try

- Complement the bits
  - Example: $7_{10} = 00111_2$ $-7_{10} = 11000_2$
  - Called *One's Complement*
  - Note: positive numbers have leading 0s, negative numbers have leadings 1s.
  - What is -00000?
    - » Answer: 11111



  - How many positive (including +0) numbers in N bits? 2^{N-1}
  - How many negative (including -0) numbers? 2^{N-1}

Shortcomings of One's complement?

- Arithmetic is less complicate than sign & magnitude.

- Still two zeros
  - 0x00000000 = $+0_{ten}$
  - 0xFFFFFFFF = -0_{ten}

- Although used for a while on some computer products, one's complement was eventually abandoned because another solution was better.

Standard Negative # Representation

- Problem is the negative mappings "overlap" with the positive ones (the two 0s). Want to shift the negative mappings left by one.
  - Solution! For negative numbers, complement, then add 1 to the result
- As with sign and magnitude, & one's complement, leading 0s → positive, leading 1s → negative
  - 000000...xxx is ≥ 0, 111111...xxx is < 0
  - except 1…1111 is -1, not -0
- This representation is *Two's Complement*
- This makes the hardware simple!

In C: short, int, long, intN_t (C99) are all signed integers.